

A Hierarchical Simple Probabilistic Population-Based Algorithm Applied to the Dynamic TSP

Edward Kupfer
Institute of Computer Science
Leipzig University, Germany

Hoang Thanh Le
Swarm Intelligence and Complex Systems Group
Institute of Computer Science
Leipzig University, Germany
lht@informatik.uni-leipzig.de

Johanna Zitt
Institute of Computer Science
Leipzig University, Germany
j.zitt@studserv.uni-leipzig.de

Ying-Chi Lin
Database Group Leipzig
Institute of Computer Science
Leipzig University, Germany
lin@informatik.uni-leipzig.de

Martin Middendorf
Swarm Intelligence and Complex Systems Group
Institute of Computer Science
Leipzig University, Germany
middendorf@informatik.uni-leipzig.de

Abstract—This work presents a Hierarchical Simple Probabilistic Population-Based algorithm (H-SPPBO) which is a hierarchical version of SPPBO. The SPPBO framework is a general scheme to design population-based algorithms for combinatorial (and other) optimization problems. The proposed H-SPPBO algorithm uses a hierarchical tree structure to make it particularly suitable for dynamic (combinatorial) optimization problems. The hierarchy is used to detect dynamic changes. H-SPPBO adapts to changes by discarding low-quality solutions whenever a change is detected. The H-SPPBO is applied to the dynamic Traveling Salesperson Problem (DTSP). The experimental results show that the hierarchical tree structure and the number of swap operations within that tree can be used as an indicator for the occurrence of dynamic changes. It is discussed how the performance of the algorithm is influenced by the threshold for detecting changes and by the number of solutions that are reset when a change is detected. The best results are obtained with medium to high thresholds and when only a certain subset of the solutions is reset.

Index Terms—ant colony optimization, hierarchical particle swarm optimization, dynamic traveling salesperson problem

I. INTRODUCTION

Simple Probabilistic Population-Based Optimization (SPPBO) [1] is a framework for combinatorial (and other) optimization algorithms that generalizes several well-known swarm intelligence algorithms, such as the Population-based Ant Colony Optimization algorithm (PACO) [2] and the Simplified Swarm Optimization algorithm (SSO) [3]. This paper extends the SPPBO framework by a hierarchical component to make them particularly suitable for solving dynamic optimization problems, i.e., problems where instances change over time. The main idea is to arrange the individual solution creating entities (SCEs, for example ants

or particles) in a tree structure such that each SCE receives information about good solutions that already have been found by its parent SCE. The positions of the SCEs within the tree change dynamically. Such a principle was already successfully used for function optimization, through the Hierarchical Particle Swarm Optimization (H-PSO) algorithm [4]. The new algorithm also extends the PACO [2] and a range of other swarm intelligence algorithms described with the SPPBO framework [1] by a hierarchical tree structure. The new algorithm is called Hierarchical Simple Probabilistic Population-Based Optimization Algorithm (H-SPPBO). It is applied in this paper to the dynamic Traveling Salesperson Problem (DTSP).

The Traveling Salesperson Problem (TSP) is a well-known optimization problem where for a given complete, undirected graph with n nodes (“cities”) and weighted edges, the problem is to find an optimal Hamiltonian cycle that minimizes the total weight of the traversed edges (e.g., a “shortest” or “fastest tour” visiting all n cities exactly once). The dynamic Traveling Salesperson Problem (DTSP) deals with dynamic changes such as changing edge weights or the removal and addition of nodes. In this work, the nodes (“cities”) have positions on a 2-dimensional space and the weight of an edge is the Euclidean distance between its two nodes. The dynamics of the DTSP consist of changes in the node positions. This type of dynamic is similar to changing travel times between cities which, for example, occur due to traffic jams or construction sites. When nodes change their positions, the traveling times to other cities change as well. For instance, nodes with changing positions could correspond to traveling persons that need to be provided with goods and resources, or moving technical vehicles needing supplies. It is well known that algorithms from the swarm intelligence field, particularly Ant Colony

algorithms [5] have been successfully used to solve the TSP.

In this work the H-SPPBO is applied to the DTSP and its performance is experimentally analyzed. It is investigated how the H-SPPBO detects changes in the DTSP and different strategies how to handle these changes are introduced. The remainder of this paper is structured as follows. An overview of related work is given in Section II. The proposed algorithm H-SPPBO is introduced in Section III. Computational results are presented in Section IV and conclusions are given in Section V.

II. RELATED WORK

The TSP is a *NP*-hard [6] combinatorial optimization problem for which a large variety of algorithms and heuristics have been proposed. Besides exact methods, such as the branch-and-cut algorithm [7], and improvement heuristics, such as 3-Opt [8] and the Lin-Kernighan heuristic [9], metaheuristics are commonly used in order to generate good solutions for the TSP in practically feasible time. Examples are biologically inspired algorithms like the evolutionary algorithm [10], [11], ant algorithms [12], particle swarm algorithms [13], and neural networks [14], [15].

Since the introduction of ant algorithms in 1992 by Marco Dorigo [16], the field of ant algorithms has lead to numerous publications, of which an overview can be found in [17]. In the PACO [2] a population of solutions is used to find a good solution. This algorithm obtained competitive results for the TSP while featuring a particularly runtime-efficient procedure for pheromone updates and pheromone evaporation that does not iterate over all entries of the pheromone matrix.

Other metaheuristics include Particle Swarm Optimization (PSO) [13] as well as its extension, the Hierarchical PSO (H-PSO) [4], [18], which were developed for function optimization problems [13]. In H-PSO, a hierarchical tree structure is used to determine the neighborhood topology for the particles and thus, how particles influence one another since the solutions of the neighbors of a particle are incorporated in the calculation of a new solution of a given particle. A variant of the PSO is the Simplified Swarm Optimization (SSO) [3]. Here, the positions of a particle describe a solution of the optimization problem. The goal of the particle swarm is to find the global minimum or maximum by exchanging their results and giving other particles hints where good solution may be positioned. The SSO as well as the PACO obtained good results on different instances of the TSP [1]. The algorithm H-SPPBO presented in this paper can be seen as a hybrid of PACO, SSO, and H-PSO that combines their advantages.

To classify and generalize the variety of population-based algorithms and their variants, the Simple Probabilistic Population Based Optimization (SPPBO) scheme was introduced in [19] to provide a unified framework for population-based metaheuristics. This was particularly demonstrated for PACO and SSO [1] on the TSP and the Quadratic Assignment Problem. The H-SPPBO algorithm introduced in this work is formulated in the SPPBO scheme so that it can be easily compared to the other algorithms. In [1], the SPPBO framework

was also applied to more recent developments of the PACO algorithm leading to new methods that achieved good results for several TSP instances.

This paper deals with the dynamic TSP (DTSP), which has been in the focus of much research in recent years, e.g. [20]–[22]. Reviews were done, for example, by Mavrouniotis et al. in 2017 [21] and by Ertenlice and Kalayci in 2018 [23]. In the DTSP, a problem instance changes at runtime. For example, the topology of the cities [24], [25] or weights (e.g. the duration or length) between cities change [26], [27]. If the time interval between changes in the TSP is sufficiently large, improvement methods such as the Lin-Kernighan [9], 2-opt [28] or 3-Opt [8] methods can be used for reoptimization. Also, the field of evolutionary dynamic optimization provides metaheuristics which can be applied on the DTSP [29], [30]. These methods are well suited to speed up the reoptimization process, if the changes are not too large [31]. They are also applicable when the dynamics occur at relatively short intervals [32]. In real world this is the case in many applications, such as in logistics [33], robotics [34], or telecommunications [35].

III. THE H-SPPBO ALGORITHM

Using the framework for Simple Probabilistic Population-Based Optimization (SPPBO) [1], with which various swarm-based algorithms such as the Population-Based Ant Colony Optimization (PACO) [20] and the Simplified Swarm Optimization (SSO) [3] can be generalized, it is possible to give a succinct description of the proposed algorithm which combines aspects of the PACO and SSO with ideas used in the Hierarchical Particle Swarm Optimization algorithm (H-PSO).

The main components of an SPPBO algorithm are a set $\mathcal{A}$ of solution creating entities (SCEs, like ants or particles), a set $\mathcal{P}$ of populations and a function *range* that assigns to an entity $A \in \mathcal{A}$ a set of populations $range(A) \subseteq \mathcal{P}$ by which A is influenced during the construction of a solution. For every population $P \in \mathcal{P}$ a weight w_P and a set of solutions for the TSP is stored. Formally, an entity $A \in \mathcal{A}$ constructs a solution as follows for the TSP. First, a random start node is chosen as the “current node” i and the term

$$\tau_{ik}(A) = \left(w_{rand} + \sum_{P \in range(A)} w_P \cdot s_{ik}(P) \right)^\alpha \cdot \eta_{ik}^\beta, \quad (1)$$

is calculated for every possible next node $k \in U$, where U is the set of unvisited nodes. w_{rand} is a parameter (weight for random selection), $s_{ik}(P)$ is the number of solutions in P that also visit the node k after the node i , the term η_{ik} is the heuristic component and α, β are parameters to weight the influence of the heuristic and probabilistic component. In this work, the heuristic component η_{ik} is chosen as the inverse distance $1/d_{ik}$ between nodes i and k . Afterwards, the probability

$$p_{ij}(A) = \frac{\tau_{ij}(A)}{\sum_{k \in U} \tau_{ik}(A)}. \quad (2)$$

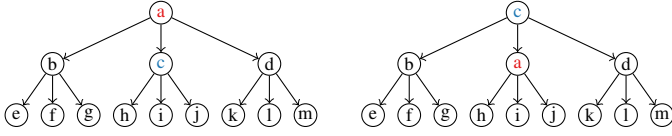


Fig. 1. Example for a swap in a ternary SCE tree with three levels. An arrow $x \rightarrow y$ indicates that the SCE x is the parent of y . In this example, the positions of the SCEs a and c in the tree are swapped.

of visiting node j after node i is calculated and the next node j is randomly selected based on this probability distribution. This procedure is repeated with the newly visited node j being the “current node” until all cities have been visited. After every SCE $A \in \mathcal{A}$ constructs a solution, the populations $P \in \mathcal{P}$ are updated with population-specific update procedures $update_P$. More specifics and different variants of the algorithm can be found in [1].

For the *Hierarchical Simple Probabilistic Population-Based Optimization algorithm* (H-SPPBO) presented in this work (shown in Algorithm 1) a hierarchical tree structure is defined between the SCEs with a “root SCE” A^* . Every SCE $A \neq A^*$ is assigned a “parent SCE” $parent(A) \in \mathcal{A}$, leading to an “SCE tree”. The basic idea is that the solutions stored by $parent(A)$ influence the “child entity” A . More precisely, the function *range* assigns to every SCE A three populations $P_{persprev}^A, P_{persbest}^A, P_{parentbest}^A$ that each contain one solution. $P_{persprev}^A$ contains the solution constructed by A in the previous iteration (personal previous solution), $P_{persbest}^A$ the personal best solution and $P_{parentbest}^A$ contains the personal best solution of the parent SCE, i.e., $P_{parentbest}^A = P_{persbest}^{parent(A)}$. For the “root SCE” A^* , we define $P_{parentbest}^{A^*} = P_{persbest}^{A^*}$. The choice of populations is similar to algorithm SPPBO-7 in [1] which was shown to obtain good results for the TSP.

Regarding the SCE tree, it is possible that an entity A switches places with its parent $parent(A)$ depending on the solution qualities of their respective personal best solutions (see Fig. 1 for an example). Thus, extending the SPPBO, the H-SPPBO has a function *range* that changes dynamically. This is done so that the upper levels of the SCE tree contain SCEs with good personal best solutions, similar to the H-PSO algorithm [4]. More precisely, a swap between A and $parent(A)$ occurs if A has a better personal best solution than $parent(A)$. This condition for swapping two entities is checked at every iteration in a top-down manner which ensures that SCEs with personal best solutions of low quality quickly descend the tree, whereas personal best solutions of high quality need to be “good” over multiple iterations in order to reach the upper levels of the tree.

For the dynamic TSP where the problem changes over time, the tree structure can also be used to detect changes, similar to [4]. Since such changes affect the quality of the personal best solutions $P_{persbest}^A$ for each entity A leading to swaps in the SCE tree, a sudden increase in the number of swaps per iteration can be used as an indicator to detect changes, e.g.,

by checking whether this number exceeds a given threshold. In this work, the threshold is chosen as a percentage θ of the number of solution creating entities $|\mathcal{A}|$. For example, if $\theta = 0.25$, the algorithm interprets an increase in the number of swaps as the occurrence of a “dynamic change” if this number exceeds 25% of the number of SCEs in the tree. The detection of changes is important since the algorithm itself does not know when changes in the instances occur.

Upon detecting that a change has occurred, the algorithm calls a change handling procedure H to adapt to the change. Since it is possible that good solutions stored in the populations can lose their good quality because of dynamic changes, the approach used in this work is to discard stored solutions by resetting some of the populations with random solutions. By doing so, the algorithm decreases its focus on the solutions found so far while increasing the exploration of new solutions in the subsequent iterations. In this work, two change handling procedures are investigated. One is to reset the personal best populations $P_{persbest}^A$ for all SCEs (abbreviated as H_{full} in the following) which is similar to resetting the algorithm. The other change handling procedure is to only reset the personal best populations of SCEs starting from the third level in the SCE tree whereas the root SCE and its child SCEs remain unchanged. This change handling procedure acts as a partial reset $H_{partial}$ that retains the “knowledge” stored in the best performing SCEs so that only low-performing SCEs increase their focus on exploration by discarding their solutions. In addition, since it is possible that a dynamic change leads to an increased number of swaps in the subsequent iterations, it is necessary to pause the detection for some time L after a change has been detected so that the handling procedure is not called multiple times for the same change. In this work, the detection is paused for $L = 5$ iterations whenever a change has been detected.

IV. COMPUTATIONAL RESULTS

In order to evaluate the proposed algorithm, the instances rat195, a280 and pr439 from the TSPLIB benchmark [36] which contain 195, 280, and 439 nodes, respectively, are used. These instances are selected since they are the same instances that are used for evaluating the SPPBO-based algorithms in [1].

A. Experimental Parameters

For the populations $P_{persprev}$ and $P_{persbest}$ the weights $w_{persprev} = 0.075$ and $w_{persbest} = 0.075$ are chosen. Furthermore, we set $w_{rand} = 1/(n-1)$ as well as $\alpha = 1$ and $\beta = 5$. The choice of these parameters is based on [1] where good results were obtained with these values. In addition, the weight $w_{parentbest}$ for the population $P_{parentbest}$ is chosen as $w_{parentbest} = 0.01$ analogous to the weight of the global best solution in [1] as the latter can be seen as a part of the former (in particular, the parent best solutions in the first two levels of the SCE tree correspond to the global best solution). For the SCE tree, a ternary tree with three levels was chosen with

Algorithm 1 H-SPPBO

```
1: initialize the populations  $P_{persprev}^A, P_{persbest}^A$  with random
   solutions for all SCEs  $A$ 
2: initialize the SCE tree randomly and set  $P_{parentbest}^A$ 
3: for  $t = 1, 2, \dots, \max\_iteration$  do
4:   for all  $A \in \mathcal{A}$  do
5:     construct a solution for SCE  $A$  using (1) and (2)
6:     update the populations  $P_{persprev}^A$  and  $P_{persbest}^A$ 
7:   end for
8:    $number\_of\_swaps = 0$ 
9:   for all  $A \in \mathcal{A}$  (in top-down order of the SCE tree) do
10:    if personal best solution of  $A$  is better than personal
       best solution of  $parent(A)$  then
11:      swap  $A$  and  $parent(A)$  in the SCE tree
12:       $number\_of\_swaps = number\_of\_swaps + 1$ 
13:    end if
14:  end for
15:  if  $number\_of\_swaps > \lceil \theta \cdot |A| \rceil$  and no change
       detected in  $L$  previous iterations then
16:    call change handling procedure  $H$ 
17:  end if
18: end for
```

a total of $|A| = 13$ nodes (similar to Fig. 1 left) so that in each iteration 13 solutions are generated and evaluated.

As for the dynamics for each TSP instance, the algorithm first runs for 2000 iterations without changes in the problem instance in order to minimize effects caused by randomness during the initial iterations and to allow the algorithm to reach a steady state. After that, $\lfloor n \cdot C\%/2 \rfloor$ pairs of distinct nodes are randomly selected and the positions of the nodes in each pair are swapped. This happens every 100 iterations so that at iterations $2000 + k \cdot 100$ for $k = 0, 1, 2, \dots$ approximately $C\%$ of the nodes change their positions. This type of dynamic change has the advantage that the length of the optimal solution does not change over time. This allows one to consistently evaluate the algorithm's performance at a given time using the *relative difference* to the optimal solution (also known as RPD), calculated as $R_x = (f(x) - opt)/opt$ where $f(x)$ is length of the current solution x and opt is the length of the optimal solution (taken from the TSPLIB benchmark) which forms a lower bound for the objective function. Recall that the changes are not known to the algorithm.

As for the parameter C , the values $C \in \{10, 25, 50\}$ are chosen which correspond to a low, medium and high intensity for dynamic changes. The thresholds θ by which the proposed algorithm detects changes in the instance are chosen as $\theta \in \{0.1, 0.25, 0.5\}$. The algorithm is run on the three instances with every value for C , θ , both change handling procedures H , and with 10 replications, over which the obtained solution quality is averaged. The runs were executed on a computer with eight 1.80-GHz cores and 16 GB RAM (each run being executed on a single core) and set to terminate after 2599 iterations, allowing the algorithm to react to 6 dynamic changes during every run.

B. Solution Quality

Fig. 2 shows the solution quality over time of the current best solution found by the algorithm for different instances, values of C , and thresholds θ . Note that only iterations 2000–2300 are shown in order to facilitate the visual comparison of the curves. It can be seen that the relative difference R_x (and thus the length of the current best solution) suddenly increases whenever the instance changes at iteration $2000 + k \cdot 100$ for $k = 0, 1, 2, \dots$, which shows that the dynamic changes in general have a negative effect on the solution quality. It can also be observed that for the same threshold θ (indicated by the same color) the algorithm performs better with the partial reset $H_{partial}$ (solid line) than with the full reset H_{full} (indicated by dashed lines) as shown by the curves reaching lower values with $H_{partial}$ in most cases (although there are exceptions, such as the instance rat195 with $C = 25$).

Especially for $\theta = 0.1$ (dark blue lines in Fig. 2) it was observed that the difference in performance at the iterations $2000 + k \cdot 100 - 1$ (i.e., after approximately 100 iterations have passed since the last change and directly before the next change in the instance) is highly significant between $H_{partial}$ and H_{full} ($p < 0.001$, using the nonparametric sign test for paired samples, paired over 54 samples resulting from 3 instances, 3 intensities C and 6 dynamic changes per run). The better results for $H_{partial}$ indicate that even for dynamic environments with low and high dynamics it is preferable to retain knowledge by storing solutions in the SCEs instead of discarding all solutions whenever a change is detected.

The curves also show that with $\theta = 0.1$ the algorithm does not perform as well as with $\theta = 0.25$ or $\theta = 0.50$. One reason for this could be that a low threshold leads to falsely detected dynamic changes and unnecessary resets in the SCE tree not giving enough time to improve the solutions obtained prior to the reset. Interestingly, for $\theta = 0.25$ or $\theta = 0.50$ the results are mixed, e.g., for instance rat195 with $C = 10$ the algorithm performs well with $\theta = 0.5$, whereas for instance pr439 and $C = 10$ there are segments where it is outperformed by $\theta = 0.25$. The parts where the algorithm performs well with $\theta = 0.5$ could also indicate that in some cases a conservative approach might be preferable that retains solutions even when dynamic changes in the instance occur.

C. Detection of Changes

In addition to the attained solution quality, it is interesting to look at how the algorithm detects changes in the instance using the number of swaps in the SCE tree. Fig. 3 shows the number of changes in the SCE tree structure over time for the case $H = H_{partial}$ for different instances, intensities C for dynamic changes and thresholds θ . Similar to Fig. 2, only the iterations 2000–2300 are shown so that the curves can be compared more easily. It can be seen that there is a sudden increase in swaps whenever a change occurs. This indicates that it is possible to use the number of swaps in the SCE tree to detect changes, similar to the Hierarchical PSO [4], but for combinatorial optimization problems. Interestingly, an increase in the intensity C of dynamic changes does not consistently

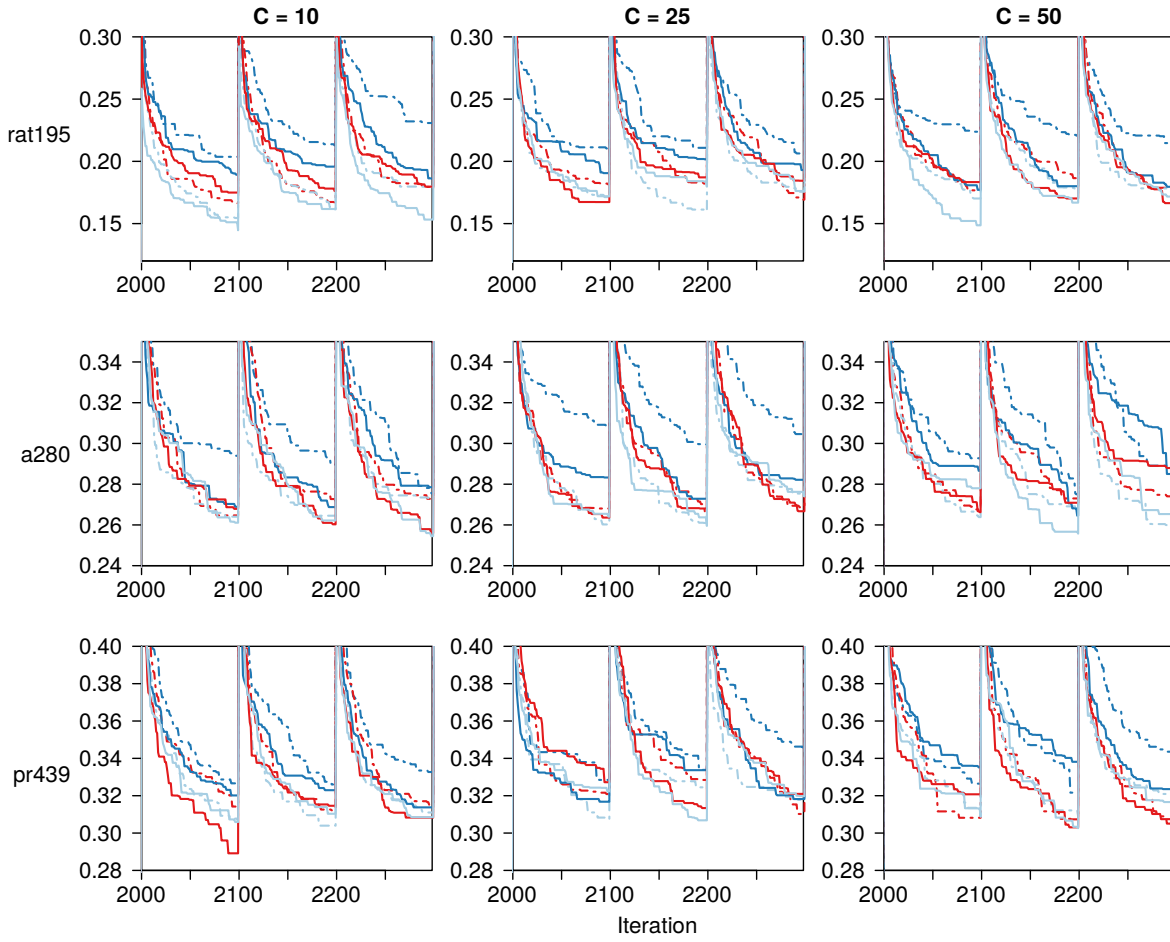


Fig. 2. Quality of the current best solution over time for different instances, different intensities C for dynamic changes and different thresholds θ , averaged over 10 runs for each configuration. The solution quality is shown as the relative difference (RPD) $R_x = (f(x) - opt)/opt$ to the optimal solution quality opt so that values close to $R_x = 0.0$ are preferable. The colors correspond to the thresholds $\theta = 0.1$, $\theta = 0.25$ and $\theta = 0.5$ used for detecting dynamic changes. Solid lines indicate that $H = H_{partial}$ is used, whereas dashed lines are used for the runs with $H = H_{full}$.

lead to more swaps in the SCE tree, but it can be observed that the number of swaps over time is increased in general for low thresholds θ , particularly for $\theta = 0.1$. This shows that frequent resets in the stored solutions due to the low threshold lead to more movement in the SCE tree in general, which in turn leads to more calls to the change handling procedure H . This can potentially be used for highly dynamic problem instances where “good” solutions can suddenly lose their good quality so that it is desirable to quickly discard solutions and focus on exploration.

Although threshold $\theta = 0.5$ and its corresponding dashed line in Fig. 3 have a high value, it should be noted that these curves only show the average values, i.e., there are single runs where the threshold is exceeded and a reaction H is triggered. Even though this potentially leads to some dynamic changes being undetected, it allows the algorithm to i) maintain its population of stored solutions during small changes and ii) only reset its solutions when unusually large changes in the problem occur. Thus, making the algorithm follow a more conservative approach that focuses on keeping and improving

the solutions found so far.

In order to investigate the accuracy with which the proposed method detects dynamic changes, two ratios $r_{correct}$ and r_{total} were used that describe how accurately changes are detected and how well unnecessary calls to the change handling procedure H (“false positives”) are avoided. The value $r_{correct}$ is the ratio between correctly detected changes and the total number of dynamic changes that occurred. Value r_{total} is the ratio between the number of calls to a change handling method H (the sum of correctly and falsely detected changes) and the total number of dynamic changes. Table I shows the results for the three instances for different intensities C , thresholds θ , and for both change handling methods $H_{partial}$ (discarding solutions in the lower part of the SCE tree, left half of the table) and H_{full} (discarding all solutions in the tree, right half of the table).

It can be seen that for the lowest threshold $\theta = 0.1$ the algorithm detects most of the changes that occur (at least 90% except for one case) using the proposed method. However, there are also many false positives, especially if the tree is

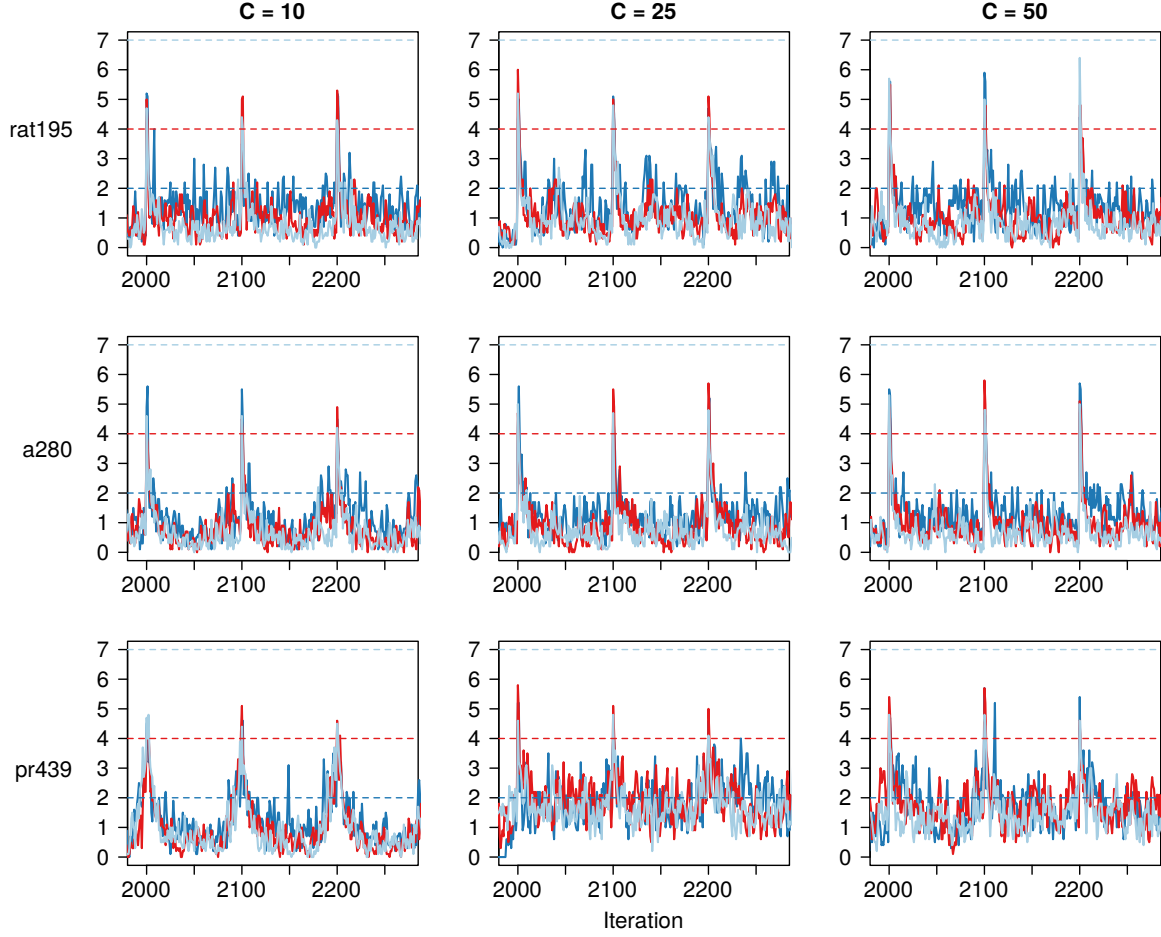


Fig. 3. Average number of swaps in the SCE tree over time with $H = H_{\text{partial}}$ for different instances, different intensities C for dynamic changes and thresholds θ , averaged over 10 runs. The colors correspond to the thresholds $\theta = 0.1$, $\theta = 0.25$ and $\theta = 0.5$ used for detecting dynamic changes, and the dashed horizontal lines indicate the number of swaps in the SCE tree that is necessary to invoke a change handling procedure H when $|A| = 13$ SCEs are used.

TABLE I

ACCURACY OF H-SPPBO REGARDING THE DETECTION OF DYNAMIC CHANGES FOR DIFFERENT INSTANCES, INTENSITIES FOR DYNAMIC CHANGES, VALUES FOR THE THRESHOLD θ AS WELL AS DIFFERENT CHANGE HANDLING METHODS H , AVERAGED OVER 10 RUNS. A DYNAMIC CHANGE IS COUNTED AS “CORRECTLY DETECTED” IF, WITHIN 5 ITERATIONS AFTER IT OCCURS, A CHANGE HANDLING METHOD H IS CALLED (WHICH HAPPENS WHEN THE NUMBER OF CHANGES IN THE TREE STRUCTURE EXCEEDS THE THRESHOLD $\theta|A|$). THE VALUE r_{correct} IS THE RATIO BETWEEN CORRECTLY DETECTED CHANGES AND THE TOTAL NUMBER OF DYNAMIC CHANGES THAT HAVE OCCURED, WHEREAS r_{total} IS THE RATIO BETWEEN THE NUMBER HOW OFTEN A CHANGE HANDLING METHOD H WAS CALLED (THE SUM OF CORRECTLY AND FALSELY DETECTED CHANGES) AND THE TOTAL NUMBER OF DYNAMIC CHANGES.

Instance	Intensity	$H = H_{\text{partial}}$ (partial reset)						$H = H_{\text{full}}$ (full reset)					
		$\theta = 0.1$		$\theta = 0.25$		$\theta = 0.5$		$\theta = 0.1$		$\theta = 0.25$		$\theta = 0.5$	
		r_{correct}	r_{total}	r_{correct}	r_{total}	r_{correct}	r_{total}	r_{correct}	r_{total}	r_{correct}	r_{total}	r_{correct}	r_{total}
rat195	$C = 10$	0.93	8.38	0.90	3.57	0.23	0.72	0.92	11.75	0.80	3.12	0.23	0.63
	$C = 25$	0.90	8.37	0.78	3.38	0.18	0.75	0.95	11.48	0.83	3.70	0.15	0.68
	$C = 50$	0.97	8.65	0.88	3.48	0.30	0.72	0.93	11.78	0.93	3.80	0.23	0.70
a280	$C = 10$	0.91	7.45	0.81	3.10	0.18	0.56	0.92	10.53	0.80	3.04	0.20	0.61
	$C = 25$	0.92	7.74	0.83	3.27	0.20	0.81	0.92	10.88	0.87	3.64	0.19	0.74
	$C = 50$	0.95	8.07	0.92	3.51	0.27	0.68	0.93	11.07	0.90	3.52	0.18	0.71
pr439	$C = 10$	0.88	7.04	0.76	3.23	0.17	0.63	0.92	9.71	0.80	3.08	0.26	0.72
	$C = 25$	0.91	8.70	0.83	4.59	0.21	1.10	0.92	11.11	0.86	4.68	0.24	1.21
	$C = 50$	0.93	8.65	0.89	4.24	0.26	1.02	0.94	10.58	0.88	3.77	0.17	0.88

fully reset ($H = H_{full}$) upon detection of a change as it calls the change handling procedure up to 11 times more often than the actual number of dynamic changes. The high values for $H = H_{full}$ when compared with $H = H_{partial}$ for $\theta = 0.1$ indicate that a full reset leads to increased movement in the SCE tree for more than 5 iterations. This leads to a higher number of calls to H_{full} due to the low threshold. Threshold $\theta = 0.5$ is so high that only a small fraction of changes is detected for both $H_{partial}$ and H_{full} , although the ratio r_{total} between the number of calls to H and the number of dynamic changes in the instance is under 1 in most cases. This means that the number of false positives is strongly reduced. It can also be interpreted in the sense that a high threshold leads to a more conservative approach where the stored solutions are not discarded until extremely large changes occur, similar to the observations made above in Fig. 3. For $\theta = 0.25$, the number of false positives is also strongly reduced (down to a factor $r_{total} \approx 3.5$ in some cases) when compared with $\theta = 0.1$ while still maintaining a fairly high accuracy ($r_{correct} > 75\%$). So this threshold can be considered to form a compromise between these two extremes.

Interestingly, with increasing intensities C for the dynamic changes a slight increase in the accuracy $r_{correct}$ can be observed in most cases. A possible explanation for this is that the increased intensity for the changes that actually occur leads to more severe changes in the qualities of the solutions stored in the SCEs which in turn leads to more swaps in the SCE tree and more detected changes. These results show that the sensitivity with which changes are detected can be adjusted with parameter θ and that the detection accuracy can be optimized by tuning the algorithm's parameters to the problem instance and its dynamics.

V. CONCLUSION

Based on the SPPBO framework that generalizes PACO, SSO and other population-based algorithms for combinatorial optimization problems, a hierarchical simple population-based algorithm H-SPPBO was presented in this work and applied to the dynamic TSP. Similar to the hierarchical PSO algorithm for continuous optimization problems [4], it uses a tree structure for the solution creating entities where the number of swaps per iteration is used to detect dynamic changes, after which it resets a number of populations in order to discard obsolete solutions and accelerate the adaptation to the changed problem. H-SPPBO extends the SPPBO by a dynamic function *range*. In this work, a full reset and a partial reset of the populations was tested. The results show that the number of swaps in the SCE tree can be used as a viable indicator for detecting changes in the problem and that the threshold with which they are detected affects the number of false positive detections. The algorithm performed better with respect to solution quality over time if the threshold is not too low. It was observed that partial resets where the algorithm retains some of the solutions found so far lead to more favorable results when compared to a full reset of all populations. For future research, the automated configuration and optimization of the parameters

to the specific problem instance and its dynamics is to be investigated. Moreover, its performance for other dynamic optimization problems and comparison to corresponding state-of-the-art algorithms will be done.

REFERENCES

- [1] Y.-C. Lin, M. Clauss, and M. Middendorf, "Simple probabilistic population-based optimization," *IEEE Transactions on Evolutionary Computation*, vol. 20, no. 2, pp. 245–262, 2015.
- [2] M. Guntsch and M. Middendorf, "A population based approach for aco," in *Applications of Evolutionary Computing*, S. Cagnoni, J. Gottlieb, E. Hart, M. Middendorf, and G. R. Raidl, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 72–81.
- [3] W.-C. Yeh, "A two-stage discrete particle swarm optimization for the problem of multiple multi-level redundancy allocation in series systems," *Expert Systems with Applications*, vol. 36, no. 5, pp. 9192–9200, 2009.
- [4] S. Janson and M. Middendorf, "A hierarchical particle swarm optimizer for noisy and dynamic environments," *Genet. Program. Evolvable Mach.*, vol. 7, no. 4, pp. 329–354, 2006.
- [5] M. Dorigo, M. Birattari, and T. Stützle, "Ant colony optimization," *IEEE Computational Intelligence Magazine*, vol. 1, no. 4, pp. 28–39, 2006.
- [6] C. H. Papadimitriou and K. Steiglitz, "Some complexity results for the traveling salesman problem," in *Proceedings of the eighth annual ACM symposium on theory of computing*, 1976, pp. 1–9.
- [7] J. E. Mitchell, "Branch-and-cut algorithms for combinatorial optimization problems," *Handbook of applied optimization*, vol. 1, pp. 65–77, 2002.
- [8] S. Lin, "Computer solutions of the traveling salesman problem," *Bell System Technical Journal*, vol. 44, no. 10, pp. 2245–2269, 1965.
- [9] S. Lin and B. W. Kernighan, "An effective heuristic algorithm for the traveling-salesman problem," *Operations research*, vol. 21, no. 2, pp. 498–516, 1973.
- [10] D. B. Fogel, "An evolutionary approach to the traveling salesman problem," *Biological Cybernetics*, vol. 60, no. 2, pp. 139–144, 1988.
- [11] A. E. Eiben, J. E. Smith *et al.*, *Introduction to evolutionary computing*. Springer, 2003, vol. 53.
- [12] M. Drigo, "The ant system: Optimization by a colony of cooperating agents," *IEEE Transactions on Systems, Man, and Cybernetics-Part B*, vol. 26, no. 1, pp. 1–13, 1996.
- [13] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95-international conference on neural networks*, vol. 4. IEEE, 1995, pp. 1942–1948.
- [14] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, "Neural combinatorial optimization with reinforcement learning," *arXiv preprint arXiv:1611.09940*, 2016.
- [15] M. Prates, P. H. Avelar, H. Lemos, L. C. Lamb, and M. Y. Vardi, "Learning to solve np-complete problems: A graph neural network for decision tsp," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 4731–4738.
- [16] M. Dorigo, V. Maniezzo, and A. Colomi, "The ant system: An autocatalytic optimizing process," *ACADEMIA*, 1991.
- [17] M. Dorigo and T. Stützle, "Ant colony optimization: overview and recent advances," *Handbook of metaheuristics*, pp. 311–351, 2019.
- [18] S. Janson and M. Middendorf, "A hierarchical particle swarm optimizer and its adaptive variant," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 35, no. 6, pp. 1272–1282, 2005.
- [19] Y.-C. Lin and M. Middendorf, "Simple probabilistic population based optimization for combinatorial optimization," in *2013 IEEE Symposium on Swarm Intelligence (SIS)*. IEEE, 2013, pp. 213–220.
- [20] M. Guntsch and M. Middendorf, "Applying population based aco to dynamic optimization problems," in *International Workshop on Ant Algorithms*. Springer, 2002, pp. 111–122.
- [21] M. Mavrouniotis, F. M. Müller, and S. Yang, "Ant colony optimization with local search for dynamic traveling salesman problems," *IEEE transactions on cybernetics*, vol. 47, no. 7, pp. 1743–1756, 2016.
- [22] C. Li, M. Yang, and L. Kang, "A new approach to solving dynamic traveling salesman problems," in *Asia-Pacific Conference on Simulated Evolution and Learning*. Springer, 2006, pp. 236–243.
- [23] O. Ertenlice and C. B. Kalayci, "A survey of swarm intelligence for portfolio optimization: Algorithms and applications," *Swarm and evolutionary computation*, vol. 39, pp. 36–52, 2018.

- [24] M. Guntsch and M. Middendorf, "Pheromone modification strategies for ant algorithms applied to dynamic tsp," in *Workshops on applications of evolutionary computation*. Springer, 2001, pp. 213–222.
- [25] D. Angus and T. Hendtlass, "Ant colony optimisation applied to a dynamically changing problem," in *International Conference on Industrial, Engineering and Other Applications of Applied Intelligent Systems*. Springer, 2002, pp. 618–627.
- [26] M. Mavrovouniotis and S. Yang, "Ant colony optimization with immigrants schemes for the dynamic travelling salesman problem with traffic factors," *Applied Soft Computing*, vol. 13, no. 10, pp. 4023–4037, 2013.
- [27] A. Simoes and E. Costa, "Chc-based algorithms for the dynamic traveling salesman problem," in *European Conference on the Applications of Evolutionary Computation*. Springer, 2011, pp. 354–363.
- [28] G. A. Croes, "A method for solving traveling-salesman problems," *Operations research*, vol. 6, no. 6, pp. 791–812, 1958.
- [29] C. Cruz, J. R. González, and D. A. Pelta, "Optimization in dynamic environments: a survey on problems, methods and measures," *Soft Computing*, vol. 15, no. 7, pp. 1427–1448, 2011.
- [30] S. Yang, Y. Jiang, and T. T. Nguyen, "Metaheuristics for dynamic combinatorial optimization problems," *IMA Journal of Management Mathematics*, vol. 24, no. 4, pp. 451–480, 2013.
- [31] Y. Jin and J. Branke, "Evolutionary optimization in uncertain environments-a survey," *IEEE Transactions on evolutionary computation*, vol. 9, no. 3, pp. 303–317, 2005.
- [32] R. Tinós, D. Whitley, and A. Howe, "Use of explicit memory in the dynamic traveling salesman problem," in *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation*, 2014, pp. 999–1006.
- [33] C. A. Silva and T. A. Runkler, "Ant colony optimization for dynamic traveling salesman problems," *ARCS 2004—Organic and pervasive computing*, 2004.
- [34] L. B. Gueta, R. Chiba, J. Ota, T. Arai, and T. Ueyama, "A practical and integrated method to optimize a manipulator-based inspection system," in *2007 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. IEEE, 2007, pp. 1911–1918.
- [35] D. Apiletti, E. Baralis, and T. Cerquitelli, "Energy-saving models for wireless sensor networks," *Knowledge and Information Systems*, vol. 28, no. 3, pp. 615–644, 2011.
- [36] G. Reinelt, "TSPLIBa traveling salesman problem library," *INFORMS Journal on Computing*, vol. 3, no. 4, pp. 376–384, November 1991.